Security Policy

Scope

Mido Deep Search is a multilingual research application composed of a static frontend, a Cloudflare Worker API, third-party research and AI providers, license verification, and local report storage.

This policy covers security issues in the repository and its application code, including:

- `index.html`
- `bacterium-lite.html`
- `worker.js`
- `wrangler.toml`
- scripts, contracts, and deployment documentation added to this repository

Third-party services such as Cloudflare, Tavily, Groq, Gumroad, GitHub Pages, and browser implementations have their own security policies and service boundaries. Report issues in those services to their respective providers when the issue is not caused by this project.

Supported versions

Security fixes are applied to the latest version on the `main` branch. Older commits and unpublished local copies are not guaranteed to receive security updates.

Version or branch	Security support
`main`	Supported
Older commits	Not guaranteed

Reporting a vulnerability

Please do not disclose suspected vulnerabilities in a public GitHub Issue, Pull Request, discussion, or social-media post before a fix or mitigation is available.

Send a private report to:

midoaistore@gmail.com

Use the subject line:

Plain Text

```
[Security Report] Mido Deep Search
```

If the issue involves an exposed secret, revoke or rotate the secret immediately where possible, then include only a redacted description in the report. Never send a live API key, admin key, license key, access token, password, or personal data by email or by opening a public issue.

What to include

A useful report should include:

1. A short description of the vulnerability and its security impact.
2. The affected file, endpoint, commit, or deployment component.
3. Clear reproduction steps or a minimal proof of concept.
4. The expected behavior and the observed behavior.
5. The environments and browsers tested.
6. Any logs or screenshots with secrets and personal data removed.
7. A suggested mitigation, if you have one.

Please keep testing safe and limited to accounts, workers, keys, and data that you own or are explicitly authorized to test.

Examples of security issues worth reporting

- Provider keys or admin credentials exposed in frontend source, URLs, browser storage, logs, screenshots, or repository history.
- Authentication or license checks that can be bypassed without authorization.
- CORS behavior that allows unauthorized origins to consume the Worker or expose protected data.
- Cross-site scripting, HTML injection, unsafe URL handling, or unsafe rendering of model output.
- Unauthorized access to admin-only endpoints or operational statistics.
- Rate-limit bypasses that materially increase provider usage or expose protected functionality.
- Sensitive information disclosed through error messages, debug endpoints, logs, or generated reports.

- Supply-chain or deployment changes that can execute untrusted code or publish secrets.

Out of scope

The following are generally not security vulnerabilities unless they create a concrete security impact:

- A feature request or visual bug.
- A report-quality issue without a security consequence.
- Public information already intentionally displayed by the application.
- Denial-of-service testing against the live Worker.
- Spam, automated traffic, or quota consumption against the live deployment.
- Issues caused solely by unsupported browsers or third-party provider outages.

Handling secrets

- Keep `TAVILY_API_KEY`, `GROQ_API_KEY`, `ADMIN_KEY`, Cloudflare tokens, and other credentials in Cloudflare Secrets or an approved secret manager.
- Do not place real credentials in `index.html`, `worker.js`, `wrangler.toml`, README files, Issues, Pull Requests, screenshots, or example files.
- Use placeholders in `.env.example` or `.dev.vars.example`.
- Do not read admin or provider secrets from URL query parameters.
- Do not persist admin or provider secrets in `localStorage` or `sessionStorage`.
- Rotate any credential that may have been exposed, even if no unauthorized use is confirmed.

Response process

After receiving a report, the maintainers will try to:

1. Acknowledge receipt privately.
2. Reproduce and assess the report.
3. Determine the affected versions and severity.
4. Prepare a fix or mitigation.
5. Coordinate disclosure when appropriate.
6. Publish a changelog entry that does not expose exploit details or sensitive data.

Response time may vary because the project depends on volunteer or small-team maintenance and third-party services.

Credit

Researchers who report a valid issue responsibly may be credited in the relevant changelog or release notes, unless they prefer to remain anonymous.